

Open/Closed Principle Explained



explain open closed principle

The **Open/Closed Principle (OCP)** is one of the five **SOLID** principles of object-oriented design. It was originally introduced by Bertrand Meyer.

🧠 Definition:

"Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification."

🔍 What does that mean?

- **Open for extension:** You should be able to add new functionality to a module or class.
- **Closed for modification:** You should not have to change the existing source code of that module/class.

This encourages writing code in a way that **new behavior can be added without altering existing tested code**, which reduces the risk of introducing bugs.

✅ Why it's useful:

- Makes your code **more maintainable**
- Supports **scalability** and **reusability**
- Helps with **unit testing** and **regression safety**

💡 Example (in Python):

❌ Without Open/Closed:

```
python

`class PaymentProcessor:
    def process(self, payment_type):
        if payment_type == "credit":
            print("Processing credit payment")
        elif payment_type == "paypal":
            print("Processing PayPal payment")
`
```

If you need to add a new payment type, you must **modify** the ``process`` method.

✅ With Open/Closed (using polymorphism):

python

```
`class PaymentMethod:
    def process(self):
        pass

class CreditCard(PaymentMethod):
    def process(self):
        print("Processing credit card payment")

class PayPal(PaymentMethod):
    def process(self):
        print("Processing PayPal payment")

class PaymentProcessor:
    def process(self, payment_method: PaymentMethod):
        payment_method.process()
```

Now you can add a new payment type (e.g., ApplePay) by creating a new class — **no need to change existing code**.

Want a real-world example in a specific language or app context?



provide the above examples in c++

Sure! Here's the same Open/Closed Principle example, but in **C++**.

✗ Without Open/Closed Principle

In this version, if you want to support a new payment type, you have to **modify** the existing `PaymentProcessor` class — which violates OCP.`

cpp

```
`#include <iostream>
#include <string>

class PaymentProcessor {
public:
    void process(const std::string& paymentType) {
        if (paymentType == "credit") {
            std::cout << "Processing credit card payment\n";
        } else if (paymentType == "paypal") {
            std::cout << "Processing PayPal payment\n";
        }
    }
};

int main() {
    PaymentProcessor processor;
    processor.process("credit");
    processor.process("paypal");
    return 0;
}
```

✓ With Open/Closed Principle

Now we use **inheritance** and **polymorphism** to allow extension without modifying existing code.

```
cpp

#include <iostream>
#include <memory>
#include <vector>

// Abstract base class
class PaymentMethod {
public:
    virtual void process() const = 0;
    virtual ~PaymentMethod() = default;
};

// Concrete class 1
class CreditCard : public PaymentMethod {
public:
    void process() const override {
        std::cout << "Processing credit card payment\n";
    }
};

// Concrete class 2
class PayPal : public PaymentMethod {
public:
    void process() const override {
        std::cout << "Processing PayPal payment\n";
    }
};

// Payment processor uses abstraction
class PaymentProcessor {
public:
    void process(const PaymentMethod& method) const {
        method.process();
    }
};

int main() {
    PaymentProcessor processor;

    CreditCard credit;
    PayPal paypal;

    processor.process(credit);
    processor.process(paypal);

    return 0;
}
```

To extend:

Just create a new class like `ApplePay`, inherit from `PaymentMethod`, and you're done — no need to touch `PaymentProcessor`.

Would you like me to add an example extension like `ApplePay`?